

課程名稱

金融資料探勘

作業名稱

金融資料探勘作業 1：運用 AI 協助建構一年份選擇權分析索引檔，並完成 Prompt 實作與優化紀錄

姓名

李秉諺

學號

611336038

組別

第三組

負責年度

西元 2023 年

繳交日期

西元 2026 年 3 月 28 日

壹、 作業目的

本作業的主要目的在於執行財務工程中的「資料前處理 (Data Preprocessing)」。透過將零散的每日大盤指數資料與選擇權結算日資料進行清理與對齊，自動化尋找符合換約條件的最近月選擇權合約，並計算出到期天數與對應的無風險利率。最終建構出一份結構化、無缺失值的「最終索引檔」，以利後續作為 Black-Scholes 模型等衍生性金融商品定價公式的標準輸入參數 (Input)。

貳、 原始資料說明

一、2023 大盤資料.csv

為 2023 年每日大盤交易紀錄，包含交易日期 (Date) 與大盤收盤指數 (So)。

二、2023 選擇權資料更新版.csv

為台灣期貨交易所之選擇權結算資料，包含各契約的「契約月份」與對應的「最後結算日」。資料內混合了月選擇權與週選擇權 (以 W 標註)。

三、無風險利率參考數據

依據台灣銀行 2023 年「一年期定期儲蓄存款 (一般金額) 機動利率」之歷史牌告利率進行設定。

四、Path_教學_0305.csv

作為最終產出檔案的欄位排序與命名格式參考範本。

參、 資料建構流程

本作業透過 Python 程式語言進行自動化資料處理，具體建構步驟如下：

一、日期格式統一與欄位改名

讀取原始大盤資料後，將 Date 欄位轉換為 datetime 格式以利後續時間差計算，並將大盤指數欄位由 So 重新命名為 S0，以符合財務模型之慣用代號與參考檔格式。

二、建立 File 欄位

依據每日交易日，利用 `strftime` 函數動態生成對應的選擇權日資料檔名，例如交易日為 2023-01-03，則新增 File 欄位內容為 `OptionsDaily_2023_01_03.csv`。

三、選擇權資料清理與排序

讀取選擇權結算資料，利用字串篩選功能排除契約月份中含有「W」的週選擇權資料，僅保留月選擇權。隨後去除重複的結算日，並依日期由小到大進行排序。

四、建立 Contract、ContractExpiryDate 與 Maturity 欄位

使用迴圈 (for loop) 逐日讀取大盤交易日，並計算該日與所有月選擇權結算日的時間差。設定換約條件為**「距離最後結算日需大於 1 天」**，抓取符合條件的最近一筆合約。將結果分別填入 Contract (契約月份)、ContractExpiryDate (最後結算日)，並計算出兩者相減的剩餘天數填入 Maturity 欄位。

五、建立 Rf (無風險利率) 欄位

針對大盤交易日進行條件判斷，對接 2023 年台銀定存機動利率。若交易日早於央行升息生效日 (2023/3/27)，則設定 Rf 為 0.01465 (1.465%)；若為 2023/3/27 (含) 之後，則設定 Rf 為 0.0159 (1.59%)。

六、格式微調與匯出

將 Date 欄位轉回 YYYY/M/D 的字串格式，並指定 utf-8-sig 編碼匯出為 CSV 檔，完成最終索引檔的建構。

肆、 最終索引檔欄位說明

欄位名稱	欄位意義	資料型態	說明與後續用途
Date	交易日期	字串	格式為 YYYY/M/D，作為每日觀察值的 Primary Key。
File	對應日資料檔名	字串	格式如 OptionsDaily_YYYY_MM_DD.csv，供後續程式自動讀取每日 Tick 或分鐘資料檔使用。
S0	大盤指數	浮點數	當日台股大盤指數，做為選擇權定價模型中的標的資產初始價格。
Contract	契約月份	字串	符合「距離結算日大於 1 天」條件的最近月選擇權合約代碼（如 202303）。
Contract ExpiryDate	最後結算日	字串	該目標契約的到期結算日期，格式為 YYYY/MM/DD。
Maturity	距離到期天數	整數	交易日距離結算日之實際天數。用於除以 365 轉換為年化到期時間，為定價公式之核心參數。

欄位名稱	欄位意義	資料型態	說明與後續用途
Rf	無風險利率	浮點數	依交易日對應之台銀定存利率（如 0.0159）。作為定價模型中計算折現值的無風險利率參數。

伍、成果展示

1	Date	File	S0	Contract	ContractExpiryDate	Maturity	Rf
2	2023/1/3	OptionsDaily_2023_01_03.csv	14,224.12	202303	2023/3/15	71	0.01465
3	2023/1/4	OptionsDaily_2023_01_04.csv	14,199.13	202303	2023/3/15	70	0.01465
4	2023/1/5	OptionsDaily_2023_01_05.csv	14,301.05	202303	2023/3/15	69	0.01465
5	2023/1/6	OptionsDaily_2023_01_06.csv	14,373.34	202303	2023/3/15	68	0.01465
6	2023/1/9	OptionsDaily_2023_01_09.csv	14,752.21	202303	2023/3/15	65	0.01465
7	2023/1/10	OptionsDaily_2023_01_10.csv	14,802.96	202303	2023/3/15	64	0.01465
8	2023/1/11	OptionsDaily_2023_01_11.csv	14,751.44	202303	2023/3/15	63	0.01465
9	2023/1/12	OptionsDaily_2023_01_12.csv	14,731.64	202303	2023/3/15	62	0.01465
10	2023/1/13	OptionsDaily_2023_01_13.csv	14,824.13	202303	2023/3/15	61	0.01465
11	2023/1/16	OptionsDaily_2023_01_16.csv	14,927.01	202303	2023/3/15	58	0.01465

陸、問題與修正

在本次資料清洗與合併的過程中，遭遇了以下幾項主要問題，並透過 AI 輔助與邏輯調整完成修正：

一、日期格式與欄位型態問題

問題：原始 CSV 檔案含有隱藏空白字元與 UTF-8 BOM 編碼問題，導致程式讀取時發生 `KeyError` 找不到欄位名稱；此外，日期原本為字串型態，無法直接進行天數相減。

修正：在讀取檔案時加入 `encoding='utf-8-sig'` 參數處理編碼，並強制使用 `df.columns.str.strip()` 清除所有欄位空白。隨後利用 `pd.to_datetime()` 將日期統一轉為時間戳記型態以利計算。

二、契約判定問題（排除週選）

問題：選擇權結算資料中混合了「週選擇權（W）」與「月選擇權」，若未排除將導致對應到錯誤的結算日。

修正：利用 Pandas 篩選功能 `~df['契約月份'].str.contains('W')`，將含有 W 的契約剔除，確保資料庫內僅保留月選擇權。

三、到期日對應與年底跨年問題

問題：選擇權具有「距離最後結算日需大於 1 天」的換約特性。初期實作時，12 月底的資料因為找不到隔年 1 月的結算日，導致 Maturity 出現空值。且使用 `.apply()` 函數時發生變數作用域錯誤 (NameError)。

修正：重新上傳包含 2024 年 1 月資料的「選擇權資料更新版」解決跨年問題；並捨棄複雜的 Pandas 函數，改用最基礎且穩定的 for 迴圈逐日判斷 `days_to_expiry > 1`，抓出最近的月合約。

四、Rf（無風險利率）合併問題

問題：2023 年台銀定存利率於 3/27 發生變動（由 1.465% 升至 1.590%），需精確對應至每一天的交易日中。

修正：撰寫條件判斷式 (if/else)，以大盤 Date 為基準，設定 `<2023-03-27` 填入 0.01465，其餘填入 0.0159，完成利率精準對接。

柒、AI Prompt 實作與優化紀錄摘要

本次實作的主要任務是利用 AI 協助撰寫 Python 程式，以進行大盤與選擇權結算資料的前處理、跨表比對、天數計算與利率合併。一開始，我給予 AI 的初始指令較為簡略，僅直白地要求「根據大盤日期對照選擇權結算日，條件是不能有 W 且大於 1 天，並算出剩餘天數」。由於指令缺乏細節與防護設定，導致 AI 產出的程式碼在 Colab 執行時頻繁出現編碼錯誤，甚至發生自訂函數無法讀取的狀況。

為了解決這些問題，我進行指令的優化，將修正重點放在「加入資料防呆機制」與「指定程式語法」上。經過多次迭代，我得出了最終的最佳 Prompt：「請讀取『2023 大盤資料』與『2023 選擇權資料更新版』，先排除契約月份有 W 的週選。接著請務必使用『for 迴圈』，以大盤 Date 為基準尋找距離結算日大於 1 天的最近月合約，並加入 utf-8-sig 編碼與 `.str.strip()` 去除欄位空白的防呆機制。最後依據日期(3/27 前為 1.465%，之後為 1.590%)新增 Rf 欄位並匯出 CSV。」透過這個結構化且條件明確的指令，我最終成功獲得了一段完全無報錯、可穩定執行的 Python 腳本，順利產出最終的索引檔。

捌、學習反思

透過這次作業，我發現 AI 雖然在幫忙寫程式和抓 bug 方面真的很厲害，但它並不是萬能的。當我遇到 Colab 一直跑出紅字錯誤（像是找不到欄位名稱，或是日期不能直接相減）時，只要把錯誤訊息直接丟給 AI，它馬上就能告訴我是因為資料裡面有「隱藏的空白鍵」，或是建議我加上防呆的程式碼，這幫我省下了很多上網找答案的時間。

不過，我也發現 AI 有時候會「把簡單的問題複雜化」，它一開始給了我一段很難懂的程式碼，結果在執行時一直出錯。後來我學到，只要在指令裡明確要求它「用最簡單的 for 迴圈來寫」，程式反而就順利跑成功了。另外，AI 其實也不太懂我們真正的「金融邏輯」，例如它不知道 12 月底的資料需要用到隔年 1 月的結算日才能換約，導致一開始算出來有空值。

總結來說，這次學到最重要的一課，就是我們自己還是要先搞懂金融常識，不能完全不動腦就依賴 AI。而且跟 AI 溝通時，指令要越具體越好，最好是把大問題拆成好幾個小步驟，一步一步讓 AI 幫忙寫，這樣才能真正順利地完成資料處理。